Using Embedded Toolchains

Keith Packard

Altus Metrum

keithp@keithp.com

Embedded Toolchains Contents

- Compiler (GCC or Clang)
- Linker (GNU linker or lld)
- C library (picolibc)
- C++ library (libstdc++ or libc++)
- Emulator (qemu or FVP)
- SWD/JTAG utility (openocd)
- Debugger (gdb or lldb)

AltusMetrum

Available Toolchains

- Debian (or Ubuntu) (GCC)
- Zephyr (GCC and LLVM)
- Arm GNU Embedded (GCC)
- Arm Toolchain for Embedded (LLVM)
- Crosstool-NG (GCC)
- Zephyr and Debian include everything you need; the others only provide the compiler, linker and libraries.

Debian (and Ubuntu)

```
$ sudo apt install picolibc-arm-none-eabi openocd qemu-system-arm
```

- Ideal if you're running Debian.
- Tracks GNU Arm Embedded sources.
- Stable version requires `--specs=picolibc.specs`
- “Batteries included”
- Supported by your humble presenter.

AltusMetrum

Zephyr

`github.com/zephyrproject-rtos/sdk-ng`

- Great for everything other than Debian.
- GCC version 14, which is not the latest.
- Both GCC and Clang available.
- "Batteries included"
- Supported by the Zephyr SDK team, including the presenter.
- Not just for Zephyr; these toolchains are usable for any embedded development. Highly recommended.

Arm GNU Toolchain

`developer.arm.com/tools-and-software/gnu-toolchain`

- Directly supported by ARM
- Picolibc extension available
  - Requires `--specs=picolibc.specs`
- Doesn't provide TLS support
- Does not include qemu or openocd

AltusMetrum

Arm Toolchain for Embedded

`github.com/arm/arm-toolchain`

- Directly supported by ARM
- Native picolibc support (and actively supported)
- Does not include qemu, openocd or a debugger (gdb/lldb)

AltusMetrum

Crosstool-NG

`crosstool-ng.github.io`

- Build a toolchain from source
- Native picolibc support, (1.8.10, not 1.8.11)
- Use the arm-picolibc-eabi sample
- Picolibc support maintained by the presenter.
- Took about an hour to build on my laptop.
- Does not include qemu, openocd or a debugger

AltusMetrum

Add Missing Components

For everything other than Zephyr and Debian, you'll need more stuff:

- qemu
  - https://www.qemu.org/download/
- openocd
  - https://github.com/openocd-org/openocd/releases (mingw32 only)
- gdb
  - https://packages.msys2.org/packages/mingw-w64-x86_64-gdb-multiarch
- fvp
  - https://developer.arm.com/tools-and-software/fixed-virtual-platforms

AltusMetrum

SWD

developer.arm.com/documentation/ihi0031/a/The-Serial-Wire-Debug-Port--SW-DP-

- Two wires instead of JTAG
- Supported by OpenOCD
- Source-level in-circuit debugging
- Altus Metrum products have a standard 4-pin connection using a Micro-MaTch footprint

AltusMetrum

Semihosting

`https://developer.arm.com/documentation/dui0471/g/Semihosting`

- Access host OS from the target
- Console and file I/O
- Supported by OpenOCD and QEMU
- Requires no chip-specific code for the target (unlike a UART or USB).
- Printf debugging during early chip bring-up.

AltusMetrum

Using GCC Toolchains

- Some require `--specs=picolibc.specs`
  - When picolibc support is added, not native
- Use `--crt0=semihost` and `–oslib=semihost`
  - Builds a working linker command
- Use `-mcpu=<cpu>` to configure the compiler for the target CPU

Using Clang Toolchains

- Picolibc support is built-in
- Use `-lcrt0-semihost` and `-lsemihost`
  - The linker figures it out
- Use `--target=<isa>-none-eabi` and `-mfloat-abi={hard,soft}`
  - Maybe there's an easier way?

AltusMetrum

C++

- Yes.
  - Including exceptions and RTTI.
- Libstdc++ or libc++
  - libstdc++ now uses stdio for all I/O
- Zephyr and Debian provide two versions:
  - -Os without exceptions/rtti for minimal size
  - -O2 with everything

Toolchain Setup Steps

1. Download and install
2. Compile and link simple hello-world for qemu with basic cortex-m target
3. Test with qemu
4. Adjust CPU and linker script for target board
5. Compile and link for target
6. Test on target

Linker Scripts

- The darkest of the toolchain arts.
- Defines where each chunk of the program goes.
  - Read-only bits to flash
  - Read-write bits to RAM
- Declares symbols needed by the C library.
  - How to initialize memory
  - Where to place the stack and heap
- Picolibc's sample scripts are a good starting point
  - Assumes one chunk of flash and one chunk of ram.

Linker Script Components

- MEMORY block describes memory regions for target processor.
  - Attributes (read/write/execute).
  - Address
- PHDRS block describes ELF attributes for chunks of memory
  - Each section gets an PHDRS name *and* a MEMORY name
- SECTIONS block maps symbols to memory
  - Each section groups symbols with matching MEMORY and PHDRS attribute
  - Defines symbols needed by C library
  - Debugging and other information sections don't actually end up on the device.
- Review the data sheet to get the MEMORY values.
- Read the C library documentation to learn what extra symbols are needed.

Demo

- Adafruit Metro M4 Express
  - Atmel ATSAMD51 processor (Cortex M4)
  - Using Segger J-Link for SWD
- Raspberry Pi Pico
  - RP2040 processor (Dual Cortex M0)
  - CMSIS-DAP for SWD

Source Code

```
#include <stdio.h>
#include <stdlib.h>


int
main(void)
{
    printf("hello, world\n");
    exit(0);
}
```

AltusMetrum

Linker Script for RP2040

```
__flash =       0x10000000;
__flash_size = 0x00200000;
__ram =         0x20000000;
__ram_size   = 0x00040000;
__stack_size = 1k;

INCLUDE picolibc.ld
```

AltusMetrum

Linker Script for Metro-M4 (GCC)

```
__flash =       0x00000000;
__flash_size = 0x00100000;
__ram =         0x20000000;
__ram_size    = 0x00010000;
__stack_size = 1k;


INCLUDE picolibc.ld
```

AltusMetrum

Linker Script for Metro-M4 (Clang)

AltusMetrum

```
__flash =       0x00001000;
__flash_size = 0x000ff000;
__ram =         0x20000000;
__ram_size   = 0x00010000;
__stack_size = 1k;

INCLUDE picolibc.ld
```

Building for Metro M4 (GCC)

```
arm-none-eabi-gcc \
   -mcpu=cortex-m4 \
   -Tmetro-m4.ld \
   --oslib=semihost \
   hello-world.c
```

Building for Metro-M4 (Clang)

```
clang \
  --target=thumbv7em-none-eabi \
  -mfloat-abi=hard \
  -Tmetro-m4.ld \
  -lsemihost \
  hello-world.c
```

Building for RP2040 (GCC)

```
arm-none-eabi-gcc \
   -mcpu=cortex-m0 \
   -Trp2040.ld \
   --oslib=semihost \
   hello-world.c
```

AltusMetrum

Building for RP2040 (Clang)

```
clang \
  --target=thumbv6m-none-eabi \
  -mfloat-abi=soft \
  -Trp2040.ld \
  -lcrt0-semihost \
  -lsemihost \
  hello-world.c
```

AltusMetrum

Testing with QEMU

- The Metro M4 memory map matches what qemu expects, so we can run that directly:

```
qemu-system-arm \
  -chardev stdio,mux=on,id=stdio0 \
  -semihosting-config enable=on,chardev=stdio0 \
  -monitor none \
  -serial none \
  -nographic \
  -machine mps2-an386,accel=tcg \
  -cpu cortex-m4 \
  -device loader,file=a.out,cpu-num=0
```

Running on Metro-M4

- Start OpenOCD in one window:

```
$ openocd \
        -f interface/jlink.cfg \
        -c 'transport select swd' \
        -c "set CPUTAPID 0x2ba01477" \
        -f target/atsame5x.cfg \
        -c 'init' \
        -c 'arm semihosting enable'
```

AltusMetrum

Running on Metro-M4

- Run GDB in another window:

```
$ gdb a.out
(gdb) tar rem :3333
(gdb) load
(gdb) c
```

Running on Raspberry Pi Pico

- Start OpenOCD in one window:

```
$ openocd \
  -f interface/cmsis-dap.cfg \
  -f target/rp2040.cfg \
  -c 'adapter speed 5000' \
  -c 'init' \
  -c 'arm semihosting enable'
```

Running on Raspberry Pi Pico

- Run GDB in another window:

```
$ gdb a.out
(gdb) tar rem :3333
(gdb) load
(gdb) c
```

Summary

- Lots of toolchains to choose from
- Need more than a compiler
- Go build stuff!

AltusMetrum